

Comprehensive Study Guide

CME 295: Transformers & LLMs — Lecture 3

LLMs, Mixture of Experts, Response Generation, Prompting, Inference Optimizations

Stanford CME 295 by Afshine & Shervine Amidi · For: Applied AI/ML Engineers

Contents

1	Topic Map	2
2	Deep-Dive Notes	3
2.1	LLM Definition and Characteristics	3
2.2	Mixture of Experts (MoE)	3
2.2.1	Motivation	3
2.2.2	Dense vs. Sparse MoE	3
2.2.3	Placement in the Transformer	4
2.2.4	Router (Gate) Architecture	4
2.2.5	Training Challenge: Routing Collapse and Auxiliary Loss	4
2.3	Response Generation: Decoding Strategies	5
2.3.1	Probability Computation Pipeline	5
2.3.2	Greedy Decoding	5
2.3.3	Beam Search	5
2.3.4	Sampling Strategies	6
2.3.5	Temperature: Full Mathematical Analysis	6
2.3.6	Guided Decoding	7
2.4	Prompting Strategies	7
2.4.1	Context Window	7
2.4.2	Prompt Anatomy	7
2.4.3	In-Context Learning (ICL)	8
2.4.4	Chain-of-Thought (CoT) Prompting	8
2.4.5	Self-Consistency	8
2.5	Inference Optimizations	9
2.5.1	KV Cache	9
2.5.2	GQA as KV Cache Reducer (revisited)	9
2.5.3	PagedAttention (Kwon et al., 2023)	9
2.5.4	Multi-head Latent Attention / Latent Attention (DeepSeek-V2, 2023)	10
2.5.5	Speculative Decoding (Chen et al., 2023)	10
2.5.6	Multi-Token Prediction (MTP, Gloeckle et al., 2024)	11
3	Related Concepts You Should Also Know	12
4	Build Projects	13
5	Explain It Back	14
6	Interview Practice Questions & Answers	15
7	Self-Assessment Test	17

Topic Map

1. LLM Overview

- 1.1 Definition: language model, scale, decoder-only architecture
- 1.2 What BERT is not an LLM — text generation requirement

2. Mixture of Experts (MoE)

- 2.1 Motivation: selective computation, FLOPs vs. parameters
- 2.2 Dense MoE vs. Sparse MoE; top- K selection
- 2.3 Placement in Transformer: FFN replaced by experts
- 2.4 Router/gate: token-level routing, per-layer gates
- 2.5 Training challenge: routing collapse and auxiliary loss
- 2.6 Noisy gating as regularization
- 2.7 Expert visualization (Mixtral)

3. Response Generation (Decoding)

- 3.1 Autoregressive next-token prediction pipeline
- 3.2 Probability computation: linear projection + softmax
- 3.3 Greedy decoding — local optimality, lack of diversity
- 3.4 Beam search — global optimality, length penalty, limitations
- 3.5 Sampling: top- K , top- P (nucleus sampling)
- 3.6 Temperature: mathematical proof, low vs. high effects
- 3.7 Guided decoding for structured outputs (JSON, FSM)

4. Prompting Strategies

- 4.1 Context length / context window terminology
- 4.2 Context rot phenomenon
- 4.3 Prompt anatomy: context, instructions, input, constraints
- 4.4 In-context learning (ICL): zero-shot vs. few-shot
- 4.5 Chain-of-Thought (CoT) prompting
- 4.6 Self-consistency: majority voting over reasoning paths

5. Inference Optimizations

- 5.1 KV Cache: motivation, what is cached, why not Q
- 5.2 GQA revisited as KV cache reducer
- 5.3 PagedAttention: fragmentation, block-based memory
- 5.4 Multi-head Latent Attention (MLA / DeepSeek-V2): low-rank KV compression
- 5.5 Speculative decoding: draft model, acceptance-rejection sampling
- 5.6 Multi-Token Prediction (MTP): embedded draft heads

Deep-Dive Notes

LLM Definition and Characteristics

Definition: Large Language Model (LLM)

An LLM is a **language model** (assigns probabilities to sequences of tokens, trained on next-token prediction) that is **large** across three axes:

- **Model size:** ≥ 1 billion parameters; modern frontier models are 70B–700B+.
- **Training data:** hundreds of billions to tens of *trillions* of tokens.
- **Compute:** hundreds to thousands of GPUs for weeks to months.

Modern LLMs are **decoder-only** Transformers (masked self-attention + FFN per layer, no encoder, no cross-attention).

BERT is NOT an LLM

By the current, well-established definition, BERT is not an LLM because it does not produce text — it is an encoder-only model for classification. LLMs must be text-in, text-out generative models.

Examples: GPT-4, LLaMA 3, Gemma, DeepSeek, Mistral, Qwen. >90% of modern LLMs are decoder-only.

Mixture of Experts (MoE)

Motivation

Motivation: Selective Computation

In a dense Transformer, every parameter participates in every forward pass. As models scale to hundreds of billions of parameters, this becomes prohibitively expensive. The key insight: **not all weights are useful for every token**. If we can route each token to a relevant subset of the network, we can increase model capacity without proportionally increasing compute per token.

Definition: FLOPs (Floating Point Operations)

A unit measuring the number of arithmetic operations (additions, multiplications) in one forward pass. FLOPs quantify compute cost. Sparse MoE models have lower FLOPs per forward pass than dense models of the same total parameter count, because only a fraction of experts are activated.

Dense vs. Sparse MoE

MoE General Formulation

Given n experts $\{E_1, \dots, E_n\}$ and a gating network G , the output for input $\mathbf{x}$ is:

$$\hat{y} = \sum_{i=1}^n G(\mathbf{x})_i \cdot E_i(\mathbf{x})$$

where $G(\mathbf{x}) \in \mathbb{R}^n$ is the gate's output (a probability distribution over experts via softmax).

- **Dense MoE:** All n experts contribute, weighted by $G(\mathbf{x})_i$. No savings in FLOPs.
- **Sparse MoE:** Only the top- K experts (by gate score) are activated. K is typically 1 or 2.

$$\hat{y} = \sum_{i \in \text{top-}K} G(\mathbf{x})_i \cdot E_i(\mathbf{x})$$

Only top- K experts perform their forward pass. FLOPs $\approx K/n \times$ dense cost.

Placement in the Transformer

Where MoE Goes: Replacing the FFN

The FFN has $O(d_{\text{model}} \cdot d_{\text{FF}})$ parameters with $d_{\text{FF}} \gg d_{\text{model}}$ — it is by far the largest sub-layer. In MoE-based LLMs, the FFN in each decoder block is replaced by n FFN experts plus a router (gate). At inference, each token is processed by only K of the n FFNs in each block. The attention layer is unchanged.

Token-Level Routing

For the sequence “A cute teddy bear is reading”: each token independently passes through the router at each layer. Token “teddy” may be routed to expert 3 in layer 1 and expert 7 in layer 2. Token “reading” might go to expert 1 in layer 1. The routing decision is made after the attention layer produces a contextualized token representation, which is then fed to the gate.

Router (Gate) Architecture

The gate G is a simple linear projection from d_{model} to n (number of experts), followed by softmax:

$$G(\mathbf{x}) = \text{softmax}(\mathbf{W}_G \mathbf{x})$$

The gate is **layer-specific** (each layer has its own $\mathbf{W}_G$) and **trained jointly** with the experts via standard backpropagation.

Training Challenge: Routing Collapse and Auxiliary Loss

Routing Collapse

A failure mode where the router converges to always selecting the same 1–2 experts for all tokens. The other experts receive no gradient and never improve, wasting capacity. Analogous to a “rich get richer” phenomenon in training.

Auxiliary Load-Balancing Loss (Switch Transformer, Fedus et al., 2021)

Add a regularization term to the training loss:

$$\mathcal{L}_{\text{aux}} = \alpha \cdot n \sum_{i=1}^n f_i \cdot P_i$$

where:

- f_i = fraction of tokens routed to expert i (computed per batch)
- P_i = average routing probability for expert i across the batch
- α = small hyperparameter (e.g., 10^{-2})

This product $f_i \cdot P_i$ is minimized when tokens are distributed uniformly across experts. The

loss penalizes unequal routing and incentivizes load balancing.

Noisy Gating

Adding Gaussian noise to gate logits before top- K selection: $G(\mathbf{x}) = \text{top-}K(\mathbf{W}_G \mathbf{x} + \epsilon)$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Allows underused experts to occasionally be selected by chance, providing gradient signal and preventing collapse. Functions similarly to dropout.

MoE Scaling Properties

Adding experts increases *total* parameters but not *active* parameters (those involved in one forward pass). This allows MoE models to be more **sample-efficient** — they reach a given quality level with less training compute than dense models. Example: Switch Transformer scaled to ~ 1 trillion parameters while keeping FLOPs comparable to a much smaller dense model.

Response Generation: Decoding Strategies

Probability Computation Pipeline

During inference, the LLM’s final decoder layer output $\mathbf{h} \in \mathbb{R}^{d_{\text{model}}}$ is projected to logits:

From Hidden State to Token Probabilities

$$\text{logits} = \mathbf{W}_{\text{out}} \mathbf{h} \in \mathbb{R}^V$$

$$P(\text{next token} = w_j) = \frac{\exp(\text{logits}_j/T)}{\sum_{k=1}^V \exp(\text{logits}_k/T)}$$

where V is vocabulary size and T is the **temperature** hyperparameter.

Greedy Decoding

At each step, select $w^* = \arg \max_j P(\text{next} = w_j)$.

Greedy Decoding Limitations

- **Deterministic:** Same input always produces same output (no diversity).
- **Locally optimal, not globally optimal:** Maximizing each step’s probability does not maximize the sequence probability. A lower-probability first token may lead to higher-probability continuations.

Beam Search

Beam Search

Maintain K candidate sequences (“beams”) simultaneously. At each step, expand all K beams by all possible next tokens ($K \times V$ candidates), then keep the top- K by cumulative log-probability:

$$\log P(\text{sequence}) = \sum_{t=1}^T \log P(w_t | w_1, \dots, w_{t-1})$$

After generating [EOS], output the beam with the highest (length-normalized) score.

Length Penalty in Beam Search

Raw cumulative log-prob penalizes longer sequences (more terms < 0 accumulate). A length penalty term $\frac{1}{T^\alpha}$ (e.g., $\alpha = 0.6-0.7$) normalizes by sequence length to prevent the model from favoring shorter outputs.

Why Beam Search Is Not Used for LLMs

- Computationally expensive: K full forward passes per step.
- Lacks diversity/creativity: still deterministic (same beam size = same output).
- Works well for machine translation (high-quality, near-unique correct answer) but poorly for open-ended generation (no single “correct” answer).

Sampling Strategies

Top- K Sampling

At each step, restrict the sampling pool to the K tokens with highest probability. Renormalize and sample. Prevents sampling very low-probability tokens while maintaining diversity. K is a hyperparameter (typical: 40–100).

Top- P (Nucleus) Sampling

At each step, find the smallest set S of tokens such that $\sum_{j \in S} P(w_j) \geq p$. Restrict sampling to S . The pool size adapts: small when the model is confident (one dominant token), large when uncertain.

Temperature: Full Mathematical Analysis

Temperature Effect: Proof

Rewrite the softmax probability for token k (the highest-logit token) by factoring out $\exp(x_k/T)$:

$$P(w_k) = \frac{\exp(x_k/T)}{\sum_j \exp(x_j/T)} = \frac{1}{1 + \sum_{j \neq k} \exp\left(\frac{x_j - x_k}{T}\right)}$$

Since $x_j - x_k \leq 0$ for all $j \neq k$:

- As $T \rightarrow 0$: $\exp((x_j - x_k)/T) \rightarrow 0$ for $j \neq k$, so $P(w_k) \rightarrow 1$ (pure greedy).
- As $T \rightarrow \infty$: $(x_j - x_k)/T \rightarrow 0$, so $\exp(\cdot) \rightarrow 1$ for all terms, giving $P(w_j) \rightarrow 1/V$ (uniform).

Table 1: Temperature Guide

Temperature	Distribution	Use Case
$T \rightarrow 0$	Spiky (near-greedy)	Deterministic tasks, code generation
$T \approx 0.7-1.0$	Balanced	General chat, balanced quality/diversity
$T > 1$	Near-uniform	Creative writing, brainstorming

Nothing in the Transformer is Stochastic Except Sampling

The entire forward pass (attention, FFN, layer norm) is deterministic. The only non-deterministic operation is *sampling from the output distribution*. Setting $T = 0$ (greedy) makes generation deterministic in theory, but in practice GPU floating-point operation or-

dering can introduce tiny numerical differences that propagate.

Guided Decoding

Guided Decoding

At each decoding step, mask out “invalid” next tokens according to a formal grammar or state machine. Only tokens that form valid continuations of the target format (e.g., JSON schema, regex) are allowed. The remaining valid tokens follow the normal sampling procedure. Implemented using: finite state machines (FSMs), context-free grammars (CFGs), or JSON schema validators applied to the token trie.

JSON Generation

After generating `{`, the only valid next tokens are string literals (property names). After generating `"first_name":`, only string values are valid. At each step, the set of permitted tokens is derived from parsing the generated sequence so far against the target schema.

Prompting Strategies

Context Window

Context Length / Context Window

The maximum number of tokens an LLM can process in a single forward pass. Also called **context size** or **window size**. All self-attention interactions happen within this window. Typical ranges: 8K (early GPT-4), 128K (LLaMA 3), 1M+ (Gemini).

Context Rot (Hong et al., 2025)

Performance on information retrieval degrades as context length grows. In the “needle in a haystack” evaluation, the model’s ability to find a specific fact buried in a large context decreases as distractors increase. More context is not always better — irrelevant context actively harms retrieval. Implication: for RAG applications, target the most relevant context chunks rather than dumping everything in.

Prompt Anatomy

A well-structured prompt typically contains four components:

1. **Context:** Sets the scene — who the model is, what situation is described.
2. **Instructions:** What the model should do (the task specification).
3. **Input:** The specific data/content to operate on.
4. **Constraints:** Output format requirements, safety guardrails, length limits.

In-Context Learning (ICL)

In-Context Learning (ICL)

The ability of an LLM to perform a new task by conditioning on examples provided in the prompt, without updating any model weights. “Learning” here refers to the model adapting its behavior based on context, not gradient-based learning.

- **Zero-shot:** Query only, no examples. Relies entirely on what the model learned during pre-training.
- **Few-shot:** k examples of (input, output) pairs precede the query. Typically improves performance by steering the model toward the desired format and task framing.

When Few-Shot Beats Zero-Shot, and When It Doesn't

Few-shot generally helps by demonstrating the target format and reducing ambiguity. However, examples *constrain* the model to the demonstrated distribution. For models with strong reasoning capabilities, a clear zero-shot instruction with explicit reasoning steps can match or exceed few-shot performance while being cheaper (fewer tokens) and more generalizable.

Chain-of-Thought (CoT) Prompting

Chain-of-Thought (Wei et al., 2022)

A prompting technique that forces the model to generate an explicit reasoning chain *before* producing the final answer. Improves performance on multi-step reasoning tasks (arithmetic, logical deduction, commonsense).

Two variants:

- **Few-shot CoT:** Provide examples of (question, reasoning, answer) triples.
 - **Zero-shot CoT:** Append “Let’s think step by step” to the prompt.
- **Interpretability benefit:** The reasoning chain is inspectable. Errors in the chain point directly to where the model went wrong (debuggable).
 - **Cost:** More tokens generated $\Rightarrow$ higher latency and cost. Acceptable trade-off for complex tasks.

Self-Consistency

Self-Consistency (Wang et al., 2022)

Sample M independent reasoning chains from the same prompt (using temperature > 0). Parse the final answer from each chain. Return the answer that appears most frequently (**majority voting**).

- Effectively an ensemble over reasoning paths.
- Latency $\approx \max(\text{latency of parallel samples})$ since all M queries run in parallel.
- Requires a way to extract and compare answers (regex, answer-at-end convention, or LLM judge).
- Empirically shown to improve accuracy on arithmetic benchmarks (GSM8K, MATH) by 5–15%.

Inference Optimizations

KV Cache

KV Caching

During autoregressive generation, computing attention for the new token t requires attending to all previous tokens $1, \dots, t-1$. This requires their key and value representations: $\mathbf{k}_i = W_K \mathbf{x}_i$ and $\mathbf{v}_i = W_V \mathbf{x}_i$.

Rather than recomputing these at every step, store (cache) them after first computation. At step t , only compute $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$ for the new token, retrieve the rest from cache, and compute attention.

Why Cache K and V but Not Q?

The query $\mathbf{q}_t$ from the *current* token is what we're computing attention for. We need $\mathbf{q}_t$ only at step t . The queries of past tokens $\mathbf{q}_1, \dots, \mathbf{q}_{t-1}$ are irrelevant at step t — attention is always from current token to all past tokens, not from past tokens to each other again.

KV Cache Size

For a model with L layers, h attention heads, key/value dimension d_k , and context length n :

$$\text{KV cache size} = 2 \times L \times h \times n \times d_k \times \text{bytes per element}$$

For LLaMA 3-70B (FP16): $2 \times 80 \times 64 \times n \times 128 \times 2 \approx 2.6$ GB for $n = 1000$ tokens. At long context ($n = 128K$), this becomes hundreds of GBs.

GQA as KV Cache Reducer (revisited)

GQA with G groups reduces the KV cache by factor h/G . For LLaMA 3-70B: $h = 64$, $G = 8 \Rightarrow 8\times$ smaller KV cache than MHA. This directly translates to supporting longer contexts within the same GPU memory budget.

PagedAttention (Kwon et al., 2023)

PagedAttention

A memory management system for KV cache, inspired by virtual memory paging in operating systems. Instead of pre-allocating a contiguous block of size (max context length) for each request, allocate memory in **fixed-size blocks** (e.g., 16 tokens per block) on demand. A lookup table maps token position indices to their physical memory block.

- Eliminates **internal fragmentation**: reserved space that will never be used (happens when you pre-allocate for max length but the actual sequence is shorter).
- Reduces **external fragmentation**: gaps between allocated regions.
- Enables **memory sharing**: multiple requests sampling from the same prompt prefix can share its KV cache blocks.
- Powers the **vLLM** inference serving framework.

Multi-head Latent Attention / Latent Attention (DeepSeek-V2, 2023)

Low-Rank KV Compression (MLA)

Instead of caching h full key vectors and h full value vectors per token per layer, compress them into a single low-dimensional latent vector $\mathbf{c} \in \mathbb{R}^{d_c}$ where $d_c \ll h \cdot d_k$:

$$\mathbf{c} = W_{\text{down}} \mathbf{x} \quad (\text{compress to latent space})$$

$$\mathbf{K} = W_{K,\text{up}} \mathbf{c}, \quad \mathbf{V} = W_{V,\text{up}} \mathbf{c} \quad (\text{decompress at use time})$$

Only $\mathbf{c}$ needs to be cached (not $\mathbf{K}$ and $\mathbf{V}$ separately). The compression matrix is **shared** across all h heads and across keys and values.

- Cache size per token: from $2h \cdot d_k$ (standard MHA) to d_c (MLA) — potentially 10–20× smaller.
- Interestingly, sharing the compression matrix also acts as a regularizer, and DeepSeek-V2 reports a slight *performance improvement* alongside the memory savings.
- The low-rank dimension d_c is a fixed design choice (hyperparameter).

Speculative Decoding (Chen et al., 2023)

Speculative Decoding

Use a small, fast **draft model** to generate k candidate tokens in parallel, then verify all k tokens in a **single forward pass** of the large **target model**. Accept or reject each token via a probabilistic scheme that guarantees the final distribution matches the target model's distribution.

Acceptance-Rejection Algorithm

Let P_i = draft model probability for token w_i ; Q_i = target model probability.

For each draft token w_i in order:

- If $Q_i(w_i) \geq P_i(w_i)$: **accept** w_i (target is at least as likely as draft).
- Else: accept w_i with probability $Q_i(w_i)/P_i(w_i)$; otherwise **reject**, re-sample from the adjusted distribution $[Q_i - P_i]_+$ (renormalized), and stop.

If all k draft tokens are accepted, sample the $(k+1)$ -th token from Q_{k+1} (obtained for free from the target's single forward pass).

Key property: The resulting distribution matches the target model's distribution exactly. Proof via total probability law.

Why This is Efficient

LLM inference is **memory-bandwidth bound**, not compute-bound. A single forward pass of the target model costs roughly the same whether it processes 1 token or k tokens (because the bottleneck is loading weights from HBM, not FLOP execution). By verifying k draft tokens in one pass, speculative decoding reduces the number of large-model forward passes by a factor close to the acceptance rate.

Multi-Token Prediction (MTP, Gloeckle et al., 2024)

Multi-Token Prediction (MTP)

A training objective that predicts k future tokens simultaneously from the final decoder representation, using k separate prediction heads. At inference time, the k heads act as the draft model (embedded within the same model), and the first (main) head acts as the target model. No separate draft model needed.

- Training: multi-token prediction loss (not next-token-only) forces the model to plan ahead.
- Inference: the $k - 1$ additional heads propose tokens, which are verified against the main head. Acceptance uses a greedy scheme (not the exact acceptance-rejection formula) because the training objective differs.
- Reported improvements: training efficiency + inference speedup + slight quality gains from the richer training signal.

Table 2: Inference Optimization Summary

Technique	Category	What it Optimizes	Key Papers / Tools
KV Cache	Exact	Avoids recomputing K, V	Standard in all LLM frameworks
GQA	Exact	Reduces KV cache size by h/G	LLaMA 3, Mistral
PagedAttention	Exact	KV cache memory waste	vLLM (Kwon et al., 2023)
Latent Attention	Approx	Low-rank KV compression	DeepSeek-V2
Speculative Decoding	Approx	Reduces large-model forward passes	Chen et al., 2023
Multi-Token Prediction	Approx	Embedded draft + richer training	Gloeckle et al., 2024

Related Concepts You Should Also Know

1. **Memory-Bound vs. Compute-Bound Inference:** For large models, GPU compute units (tensor cores) are underutilized because the bottleneck is loading weights from High-Bandwidth Memory (HBM). Each forward pass must reload billions of parameters. This is why batching and KV caching help more than faster GPUs alone.
2. **Retrieval-Augmented Generation (RAG):** A paradigm where relevant documents are retrieved from an external knowledge base and inserted into the LLM's context window. Context rot makes it critical to retrieve precise, relevant documents rather than large chunks.
3. **FlashAttention (Dao et al., 2022):** A hardware-aware exact attention algorithm that tiles the QK^T computation to avoid materializing the full $n \times n$ matrix in HBM. Achieves $O(n)$ memory and 2–4× speedup. Complementary to KV caching and GQA.
4. **Mixture of Experts in Production — Mixtral 8×7B:** Mistral's open-source MoE model with 8 experts per FFN layer, $K = 2$ active, giving effective 12.9B active parameters (vs. 46.7B total). Outperforms LLaMA 2-70B on most benchmarks with 6× lower active parameter count.
5. **Quantization (INT8, INT4):** Representing model weights (and/or activations) in lower precision. INT8 quantization halves memory and typically doubles throughput with minimal quality loss. Often applied post-training (PTQ). Orthogonal to and composable with KV caching and speculative decoding.
6. **Perplexity vs. Quality:** A model optimized for low perplexity (next-token prediction) produces text that is predictable but not necessarily helpful or creative. This is why instruction-tuning and RLHF are needed after pre-training (covered in later lectures).
7. **GSM8K / MATH Benchmarks:** Standard benchmarks for evaluating reasoning capabilities. CoT and self-consistency substantially improve scores on these. GSM8K: 8,500 grade-school math problems. MATH: competition mathematics.
8. **vLLM:** Open-source LLM serving framework built around PagedAttention. Standard tool for high-throughput LLM inference in production.
9. **Outlines / Guidance / LMQL:** Libraries implementing guided decoding with FSM-based or grammar-based token masking. Useful for building reliable structured output pipelines.
10. **Exposure Bias (Teacher Forcing vs. Inference):** At training time, the decoder always receives ground-truth previous tokens (teacher forcing). At inference time, it receives its own generated tokens. This train/test mismatch can cause error accumulation. Techniques like scheduled sampling or RLHF partially address this.

Build Projects

1. **Temperature and Sampling Explorer** (*2–3 hours, good pedagogical value*)
Using HuggingFace `transformers`, load a small decoder-only model (e.g., GPT-2 or Mistral-7B via Ollama). Build a Streamlit app that lets you: (a) input a prompt, (b) tune temperature, top- K , and top- P , (c) generate 5 completions and display them. Add a mode that shows the full token probability distribution at each step as a bar chart. Visually prove the spiky vs. uniform distribution effect of temperature.
2. **MoE Routing Visualizer** (*4–6 hours, portfolio signal*)
Load Mixtral-8x7B (or a smaller MoE model like Mixtral-8x7B-Instruct via Ollama). For a given input text, extract the router's gate probabilities at each layer and for each token. Build a color-coded heatmap (token $\times$ layer matrix, color = expert ID selected) similar to the Mistral paper's visualization. This directly demonstrates understanding of sparse MoE routing and produces a memorable visual artifact.
3. **Speculative Decoding from Scratch** (*6–10 hours, highest signal*)
Implement speculative decoding using a small model (e.g., GPT-2 as draft) and a slightly larger model (e.g., GPT-2-medium as target). Implement the acceptance-rejection sampling algorithm exactly as described. Measure: (a) acceptance rate at different temperatures, (b) effective tokens per large-model forward pass vs. baseline, (c) correctness — verify that the output distribution matches the target model's distribution via KL divergence estimation. Write this up as a detailed notebook. This is a direct interview signal for inference engineering roles at any major AI lab.

Explain It Back

1. Explain to a non-ML engineer why a Mixture of Experts model with 100 experts and 1 trillion total parameters can run almost as fast as a 10-billion-parameter dense model. Use the room-with-experts metaphor and define “active parameters” in your explanation.
2. A teammate proposes always using temperature $T = 0$ in production to get consistent, deterministic outputs for your customer-facing chatbot. What do you tell them? Cover: what $T = 0$ does mathematically, why it may not be strictly deterministic in practice, and when you *would* recommend low temperature vs. higher temperature.
3. Walk through the full token generation process for a decoder-only LLM, starting from an input prompt. Include: tokenization, the forward pass through decoder blocks, the linear+softmax output layer, and the top- P sampling step.
4. A coworker says: “We should just increase the context window to 10 million tokens so the model can see everything.” What concerns do you raise? Reference context rot, KV cache memory costs, and computational complexity.

Interview Practice Questions & Answers

L1 — What is the difference between dense MoE and sparse MoE?

Answer: In a dense MoE, all n experts contribute to the output, weighted by the gate probabilities. Every expert's forward pass is computed every time — no FLOPs savings. In a sparse MoE, only the top- K experts (by gate score) are activated. Only those K experts compute their forward pass. This allows the total parameter count to scale (by adding more experts) while keeping the FLOPs per token constant at roughly K/n of the dense model's cost. In practice, $K \in \{1, 2\}$ for most sparse MoE LLMs.

L1 — What is the KV cache and why doesn't it cache queries?

Answer: The KV cache stores the key and value matrices computed for all previously generated tokens. At each new decoding step, only the current token's Q, K, V are computed fresh; K and V for all past tokens are retrieved from cache. Queries of past tokens are not cached because they are irrelevant at the current step — we only need the current token's query to compute attention over all past keys and values. Caching past queries would waste memory without any computational benefit.

L2 — Prove mathematically that low temperature produces greedy-like behavior.

Answer: Let x_k be the highest logit. Factor $\exp(x_k/T)$ from numerator and denominator:

$$P(w_k) = \frac{1}{1 + \sum_{j \neq k} \exp\left(\frac{x_j - x_k}{T}\right)}$$

Since $x_j \leq x_k$, the exponents $(x_j - x_k)/T \leq 0$. As $T \rightarrow 0$: $(x_j - x_k)/T \rightarrow -\infty$ for $j \neq k$, so each $\exp(\cdot) \rightarrow 0$. Therefore $P(w_k) \rightarrow 1/(1 + 0) = 1$. All probability concentrates on the highest-logit token.

L2 — Explain routing collapse and how the auxiliary loss addresses it.

Answer: Routing collapse occurs when the router consistently selects the same 1–2 experts for all tokens. These experts receive all gradients and improve; unused experts stagnate and receive no gradient, wasting capacity. The auxiliary load-balancing loss $\mathcal{L}_{\text{aux}} = \alpha n \sum_i f_i P_i$ penalizes unequal routing: f_i is the fraction of tokens routed to expert i and P_i is the average probability assigned to expert i . This product is minimized when both distributions are uniform. Adding this to the main loss creates a gradient signal that incentivizes the router to distribute tokens more evenly across all experts.

L2 — What is the difference between top- K and top- P (nucleus) sampling?

Answer: Top- K always samples from exactly K tokens regardless of how the probability mass is distributed. If the model is very confident (one token has 95% probability), top- $K = 10$ still forces 9 other low-probability tokens into the pool. If the model is very uncertain, top- K may still cut off a large portion of valid candidates. Top- P (nucleus sampling) adapts the pool size: it takes the smallest set of tokens whose cumulative probability exceeds p . When the model is confident, this pool is small (perhaps 1–2 tokens). When uncertain, the pool is larger. Top- P is generally considered more principled because it matches pool size to model confidence.

L3 — How does speculative decoding maintain the target model’s output distribution?

Answer: The acceptance-rejection scheme is a form of rejection sampling. For each draft token w_i : if $Q(w_i) \geq P(w_i)$, accept (the target model is at least as likely to produce this token as the draft). Otherwise, accept with probability $Q(w_i)/P(w_i)$ and reject otherwise. On rejection, sample from the modified distribution $[Q - P]_+$ (positive parts, renormalized). Starting from the law of total probability, one can show that the marginal distribution of accepted tokens equals Q (the target distribution). This is a key result: speculative decoding produces *exactly* the same distribution as the target model alone, just faster.

L3 — Why is LLM inference memory-bandwidth bound rather than compute-bound?

Answer: Each forward pass must load every model weight from HBM (GPU High-Bandwidth Memory) through the memory bus, regardless of whether we’re generating 1 token or k tokens. A typical large model has weights of $O(10\text{s of GB})$, and the memory bus operates at $\sim 2\text{--}3\text{ TB/s}$. For a 70B model in FP16 ($\sim 140\text{ GB}$), loading weights takes $\sim 50\text{ms}$ — far exceeding the time the tensor cores spend computing ($\sim 5\text{ms}$). This is why: (1) batching multiple requests together improves GPU utilization, (2) quantization (smaller weight sizes) directly reduces memory bandwidth demand, and (3) speculative decoding helps by processing k tokens per large-model pass rather than one.

L1 — What is Chain-of-Thought prompting and why does it improve performance?

Answer: Chain-of-Thought (CoT) prompting instructs the model to generate an explicit reasoning chain before its final answer. This improves performance because: (1) complex problems require intermediate steps; forcing the model to write them out makes each step a simpler sub-problem, (2) the generated reasoning tokens serve as working memory, allowing the model to reference earlier steps when solving later ones, (3) it forces the model to commit to a structured approach before outputting an answer. The trade-off is latency and cost — more tokens are generated. CoT is most effective for multi-step arithmetic, logical deduction, and commonsense reasoning tasks.

Self-Assessment Test

Scoring: 16–20 = solid mastery; 12–15 = revisit weak spots; <12 = re-study.

Multiple Choice (1 point each)

Q1. In a sparse MoE with $n = 16$ experts and $K = 2$, what fraction of expert FFN parameters are active during one forward pass?

- (a) $1/16$ (b) $1/8$ (c) $2/16$ (d) All of them

Q2. What does routing collapse mean?

- (a) The router crashes due to hardware failure (b) All tokens are routed to the same 1–2 experts, starving others (c) The model routes tokens to the wrong layer (d) Experts stop updating during training

Q3. Setting temperature $T = 1.5$ (high) will:

- (a) Make the model faster at generation (b) Make the output more deterministic and focused (c) Produce a more uniform probability distribution, increasing creativity (d) Disable sampling entirely

Q4. In speculative decoding, the draft model is used to:

- (a) Replace the target model for faster inference (b) Generate candidate tokens that the target model then verifies in one pass (c) Score the target model’s outputs for quality (d) Fine-tune the target model

Q5. PagedAttention addresses which specific problem?

- (a) Slow attention computation for long sequences (b) Memory fragmentation and waste in KV cache storage (c) Routing collapse in MoE models (d) Incorrect token probabilities at low temperature

Q6. What is “context rot”?

- (a) A bug causing the context window to shrink (b) Degraded retrieval performance as context length grows due to distractors (c) Tokens being dropped when the context is too long (d) The model forgetting earlier tokens due to vanishing gradients

Q7. In Multi-Token Prediction (MTP), the “draft model” is:

- (a) A separately trained smaller model (b) The additional prediction heads embedded in the same model (c) The top- P sampling procedure (d) The KV cache

Q8. In top- P sampling with $p = 0.9$, the model is very confident (one token has 95% probability). How many tokens are in the sampling pool?

- (a) All V vocabulary tokens (b) Exactly $p \times V$ tokens (c) Just 1 token (cumulative ≥ 0.9 after first) (d) The top- K tokens for fixed K

Short Answer (1.5 points each)

Q9. Name the four standard components of a well-structured LLM prompt and give one concrete example of each.

Q10. Write the softmax probability formula with temperature and explain what happens to the probability distribution as $T \rightarrow \infty$.

Q11. Explain in two sentences why LLM inference is memory-bandwidth bound, and what implication this has for speculative decoding’s efficiency.

Q12. What is the key difference between few-shot and zero-shot in-context learning? When might zero-shot outperform few-shot?

Q13. In Multi-head Latent Attention (MLA/DeepSeek), what single quantity is cached instead of the full K and V matrices? Why does this reduce memory?

Q14. Describe the self-consistency technique in 3 sentences. What task types benefit most from it?

Scenario (2 points)

Q15. You're deploying an LLM API serving 1,000 concurrent users, each with conversations averaging 8,000 tokens. Your GPU has 80GB HBM. Your model is LLaMA 3-70B in FP16 (≈ 140 GB weights — needs 2 GPUs). Describe three inference optimization techniques you would apply to maximize throughput and minimize latency, referencing the specific techniques from this lecture. For each, explain what it optimizes and what trade-off (if any) it involves.

Answer Key

- Q1.** (c) $2/16 = 1/8$ of the experts (equivalently K/n) are active.
- Q2.** (b) The router always selects the same experts; other experts receive no gradient and remain undertrained.
- Q3.** (c) High temperature flattens the probability distribution toward uniform, increasing diversity/creativity.
- Q4.** (b) The draft model proposes k candidate tokens, which the target model verifies in a single forward pass.
- Q5.** (b) PagedAttention reduces internal and external fragmentation in KV cache memory allocation.
- Q6.** (b) As context length grows, the model’s ability to retrieve specific information degrades due to distractors.
- Q7.** (b) The additional prediction heads embedded within the same model serve as the draft model.
- Q8.** (c) Just 1 token — its probability alone (95%) exceeds the threshold $p = 0.9$.
- Q9.** (1) *Context*: “You are a teddy bear expert.” (2) *Instructions*: “Write a bedtime story.” (3) *Input*: “Location: Country of Teddy Bears.” (4) *Constraints*: “Must be under 200 words and suitable for ages 3–5.”
- Q10.** $P(w_j) = \exp(x_j/T) / \sum_k \exp(x_k/T)$. As $T \rightarrow \infty$: $(x_j - x_k)/T \rightarrow 0$ for all j, k , so $\exp(\cdot) \rightarrow 1$ for all terms, giving $P(w_j) \rightarrow 1/V$ — a uniform distribution over the entire vocabulary.
- Q11.** Loading model weights from HBM takes far longer than the actual tensor operations, making the memory bus the bottleneck. Speculative decoding helps because the target model’s per-pass cost is roughly constant whether processing 1 or k tokens, so verifying k draft tokens costs the same as generating 1 token — amortizing the memory access cost over k generated tokens.
- Q12.** Few-shot provides (input, output) examples in the prompt; zero-shot only provides the instruction. Zero-shot can outperform few-shot when: the examples constrain the model to a narrow distribution that doesn’t generalize to the test input; or when the model has strong enough reasoning capabilities that explicit step-by-step instructions (zero-shot CoT) outperform mimicking a few examples.
- Q13.** A single compressed latent vector $\mathbf{c} \in \mathbb{R}^{d_c}$ (where $d_c \ll h \cdot d_k$) is cached. Instead of storing h key vectors and h value vectors separately (total $2h$ vectors of dimension d_k), only the shared low-dimensional projection is stored, reducing cache size by up to $2h \cdot d_k / d_c$ — potentially 10–20 $\times$.
- Q14.** Self-consistency samples M independent reasoning chains from the same prompt using temperature > 0 . It then parses the final answer from each chain and returns the most frequently occurring answer (majority vote). It is most effective for tasks with a unique correct answer that can be extracted from diverse reasoning paths: arithmetic, logical deduction, and structured commonsense reasoning.
- Q15.** (1) **KV Caching + GQA**: Apply GQA (e.g., 8 KV groups instead of 64 heads) to reduce KV cache by 8 $\times$. With 80GB per GPU and 140GB weights, the remaining ~ 20 GB per GPU is for KV cache. GQA lets you serve longer contexts within this budget. Trade-off: slight quality loss vs. MHA (typically negligible). (2) **PagedAttention (vLLM)**: Use vLLM as the serving framework. PagedAttention eliminates KV cache fragmentation, allowing much higher

concurrent user counts with the same GPU memory. Trade-off: implementation complexity; use vLLM out of the box. **(3) Speculative Decoding:** Use a small 7B model (same family, e.g., LLaMA 3-8B) as draft. For typical chat conversations, acceptance rates of 60–80% yield 2–3× effective token throughput increase from the large model. Trade-off: requires loading a second (small) model, consuming additional GPU memory; and the speedup depends on draft/target agreement (varies by task).

End of Study Guide — Lecture 3

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.